

BSM-308 System Programming

Sakarya University-Computer Engineering

Main topics

The main topics of the course can be listed as follows:

- Basic information about C programming language
- C Pointers, malloc, strings, etc...
- Files and directory files, Signals
- Links (links), Shell forwarding, Shell Script
- Reading / writing from file (File I / O), system calls and buffer usage
- System calls and in / out
- Symbolic language (Assembly) (local variables, functions, branching)
- Processes (processes) and related system calls (fork, exec, dup, pipe)
- Interprocess communication, signals

Operating system

You can use the Linux-based Ubuntu operating system to perform the course-related applications.

Download Ubuntu Desktop

The open source desktop operating system that powers millions of PCs and laptops around the world. Find out more about Ubuntu's features and how we support developers and organisations below.

[Discover Ubuntu Desktop](#)[Check out the blog ›](#)

Ubuntu 24.04.3 LTS

The latest LTS version of Ubuntu, for desktop PCs and laptops. LTS stands for long-term support — which means five years of free security and maintenance updates, extended up to 12 years with [Ubuntu Pro](#).

Intel or AMD 64-bit architecture

[Download](#)

5.9GB

For other versions of Ubuntu Desktop including torrents, the network installer, a list of local mirrors and past releases [check out our alternative downloads](#).

[What's new](#)[System requirements](#)[How to install](#)

<https://www.ubuntu.com/download/desktop>

Evaluation

Evaluation System	
Semester Studies	Contribution Rate
1. Ara Sınav	60
1. Kısa Sınav	15
1. Proje / Tasarım	25
Total	100
1. Yıl İçinin Başarıya	60
1. Final	40
Total	100

CLASS GOALS

1. To become good general-purpose C programmers

This means that when you see a problem that needs to be coded, you can envision the overall structure of the solution, and then set about solving it in an ordered way, using the appropriate data structures where necessary.

2. To become decent Unix hackers

This means that you know how to make use of the many tools that Unix provides, be it commands, library calls or system calls. This also means that you understand the model of computation that Unix presents.

3. To become well-versed in the language of systems programming

If you finish this course with a decent grade (B or better), then my goal is to ensure that you can talk systems. This means that when you go out to the job market or to graduate school, when anyone wants to talk systems with you, you not only understand what they are saying, but can communicate back with them in a common language. This also means that if you have to read research papers in systems, you can do so without a huge amount of difficulty.

4. To understand basic uniprocessor organization.

This means to know pretty much what goes on from the time that a computer gets turned on, to writing and executing programs in a multi-user environment.

Course Resources

← → ↻ 🔍 web.eecs.utk.edu/~jplank/plank/dasses/cs360/

🗄️ | 🌐 Sakarya Üniversitesi 📁 Save to Mendeley 📁 veritabanı 📁 ns3 📁 akademik-makale 📁 ns3tutorial 📁 networkcode 📁 makaletarama

CS360/367 -- Systems Programming, Spring 2024 -- Course Information / Syllabus

Lecture: 12:55 - 2:10 PM - 415 Buehler Hall

Professor: [James S. Plank](#)

MK320. jplank@utk.edu. Office hours by appointment only.

← → ↻ 🔍 web.eecs.utk.edu/~huangj/cs360/

🗄️ | 🌐 Sakarya Üniversitesi 📁 Save to Mendeley 📁 veritabanı 📁 ns3 📁 akademik-makale

CS360 -- Systems Programming

[Jian Huang](#) --- Fall 2022

CS360 LINKS

- [General Information](#)
- [Syllabus](#)
- [Homeworks](#)
- [Labs](#)
- [Lecture Notes](#)
- [The Lab Home Page](#) (maintained by the TA's)

http://web.eecs.utk.edu/~jplank/plank/dasses/cs360/lecture_notes.html

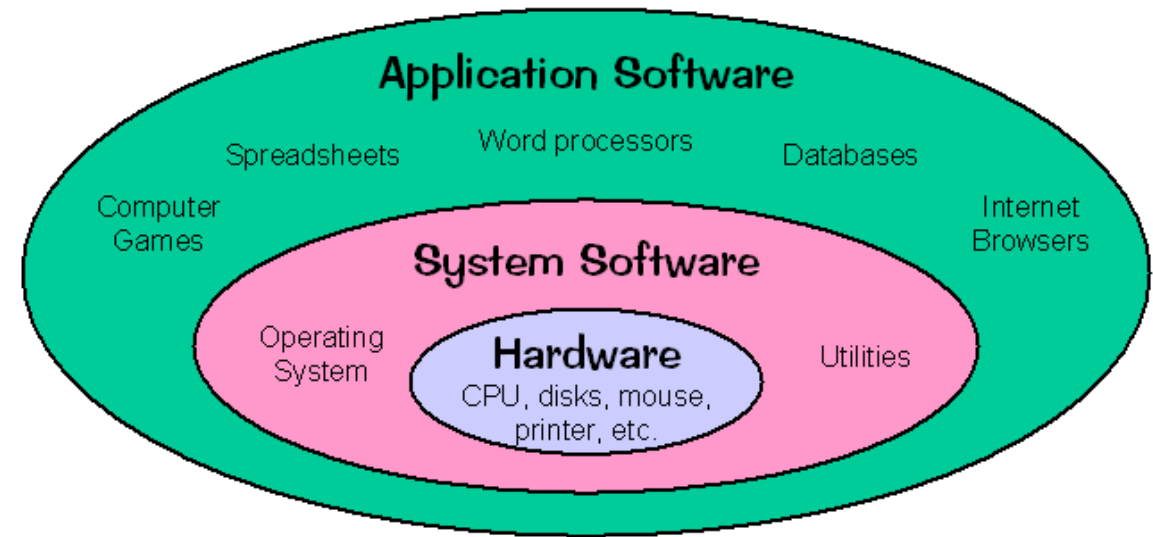
http://web.eecs.utk.edu/~huangj/cs360/lecture_notes.html

What is System programming?

- ❑ System programming : **is the use of system tools for program development.**
- ❑ In addition to programming information, it requires knowledge of computer architecture and operating system.
- ❑ While services are provided directly to the user through application programs, services are provided to application programs through system programs.
- ❑ In system programming, programs that interact with operating system services are written.

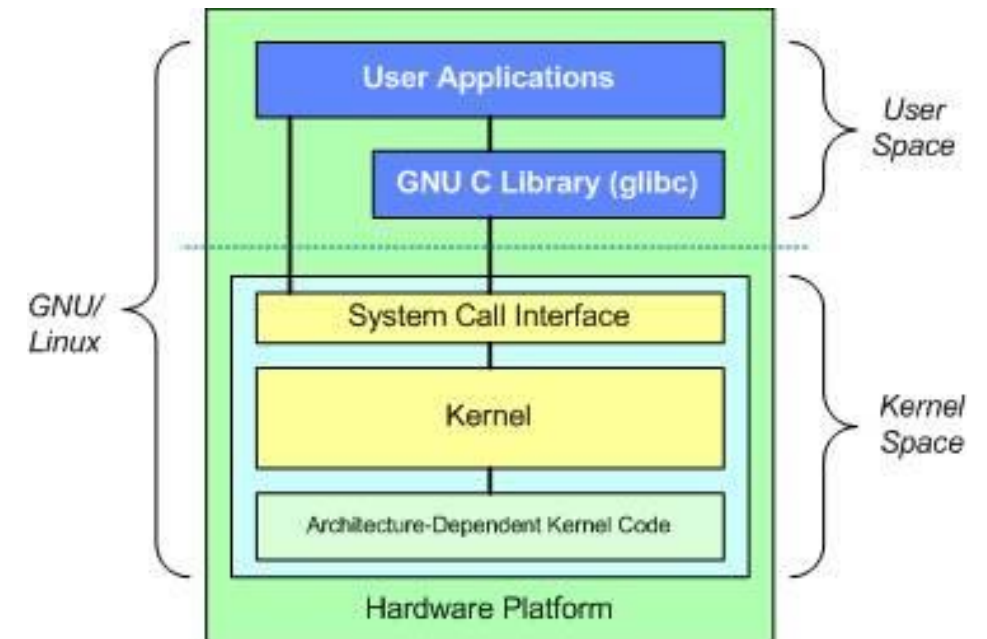
System programming

- ❑ Computer; It can be divided into three main parts: hardware, system software and application software.
- ❑ It is used to run hardware and system software application programs.
- ❑ System software creates an abstraction layer between hardware and application software.



System programming

- With the abstraction layer created by system libraries, we can use a function without knowing the details of the hardware.
- For example, the application can be used on that system as long as a system includes similar libraries, such as the GNU C library seen in the Linux architecture provided alongside.
- The use of system tools enables the creation of standards so that the developed programs are easily transferred to other computers.



Why C in system programming

- ❑ Reasons for using C in system programming:
C program; It is the basis of the modern computer by using in applications such as operating systems, device drivers, network servers.
- ❑ C programming has the least abstraction. Therefore, it is closer to the hardware.
- ❑ Many C statements can be converted directly to machine code.
- ❑ In C programming, memory can be accessed via pointers, thus enabling access to parts of the system.

Why Linux in system programming?

- ❑ Because it is open source, Linux based operating system is preferred in system programming lesson.
- ❑ Details about the operation of the Linux-based operating system can be examined. Parts can be added or replaced.
- ❑ It is not a closed system because it is not commercial.

Make macro

make is a build automation tool in Linux used to compile and manage projects, especially those written in C/C++. It uses a file called Makefile, which defines rules, dependencies, and commands. With make, you can automatically recompile only the parts of a program that have changed, saving time and effort.

❑ Macros are used in **Makefile** files to avoid some long typing. A macro is sort of like a symbolic constant in C. The basic syntax of a macro is :

```
name : values\s Example usage: ${MAKRO_NAME}
```

❑ If we used a macro, our file would be ;

```
LIBCDIR = /usr/lib/
```

```
COMPILER = cc
```

```
OBJS = main.o main.o io.o data.o
```

```
EXE = project
```

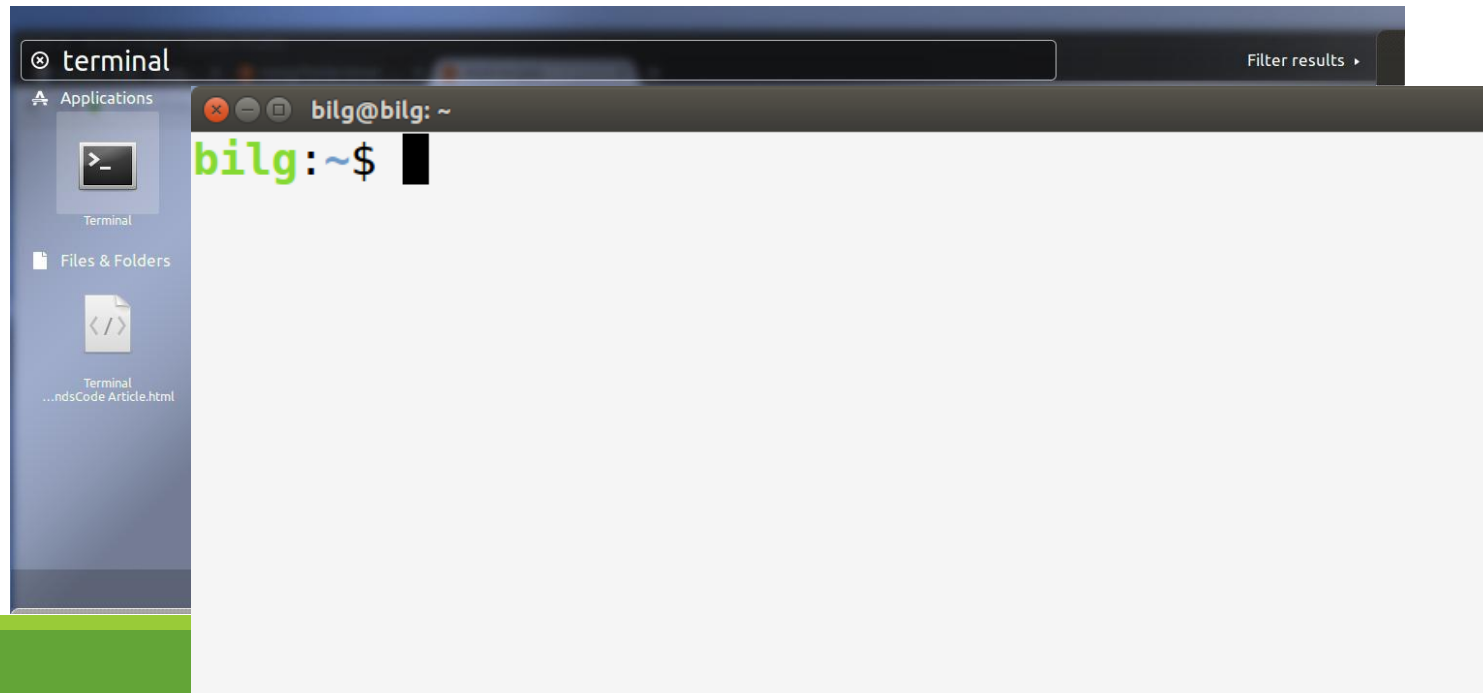
```
${EXE} : ${OBJS} ${LIBCDIR}libc.a
```

```
${COMPILER} -o ${EXE} ${OBJS} ${LIBCDIR}libc.a
```

```
@echo == Successful compilation!... ==
```

Terminal

The **Linux terminal** is a text-based interface that allows users to interact with the operating system by typing commands. It provides direct access to system functions such as file management, process control, software installation, and networking. Unlike graphical interfaces, the terminal is powerful, flexible, and often faster for advanced tasks. Common shells used in the Linux terminal include **Bash**, **Zsh**, and **Fish**.



Terminal

The **ls** command in Linux is used to **list files and directories** in the terminal. By default, it shows the contents of the current directory.

Common options:

- `ls -l` → shows detailed information (permissions, size, date).
- `ls -a` → shows hidden files (starting with `.`).
- `ls -lh` → shows file sizes in a human-readable format.

```
bilg@bilg: ~  
bilg:~$ ls  
Desktop          Program  
Documents        Public  
Downloads        Sysprog  
examples.desktop system.html  
Music            Templates  
Pictures         Videos  
PlayOnLinux's virtual drives websites  
bilg:~$
```

Terminal

The **man** command in Linux is used to **display the manual pages** of other commands, programs, or system calls.

```
bilg@bilg: ~  
bilg:~$ man ls
```

```
bilg@bilg: ~  
LS(1)                                User Commands                                LS(1)  
  
NAME  
    ls - list directory contents  
  
SYNOPSIS  
    ls [OPTION]... [FILE]...  
  
DESCRIPTION  
    List information about the FILES (the current  
    directory by default). Sort entries alphabeti-  
Manual page ls(1) line 1 (press h for help or q to quit)
```

- **ls -a** → Shows all files, including hidden ones (. files).
- **ls -l** → Long listing format (permissions, owner, size, date).
- **ls -h** → Human-readable file sizes (e.g., KB, MB).
- **ls -lh** → Long format + human-readable sizes.
- **ls -la** → Long format + all files, including hidden.
- **ls -t** → Sorts by modification time (newest first).
- **ls -r** → Reverses the sorting order.
- **ls -R** → Lists directories and their contents recursively.
- **ls -S** → Sorts by file size (largest first).
- **ls -d */** → Shows only directories.

```
bilg@bilg: ~  
short options too.  
  
-a, --all  
    do not ignore entries starting with .  
  
-A, --almost-all  
    do not list implied . and ..  
  
--author  
    with -l, print the author of each file  
  
-b, --escape  
    print C-style escapes for nongraphic characters  
  
--block-size=SIZE  
Manual page ls(1) line 15 (press h for help or q to quit)
```

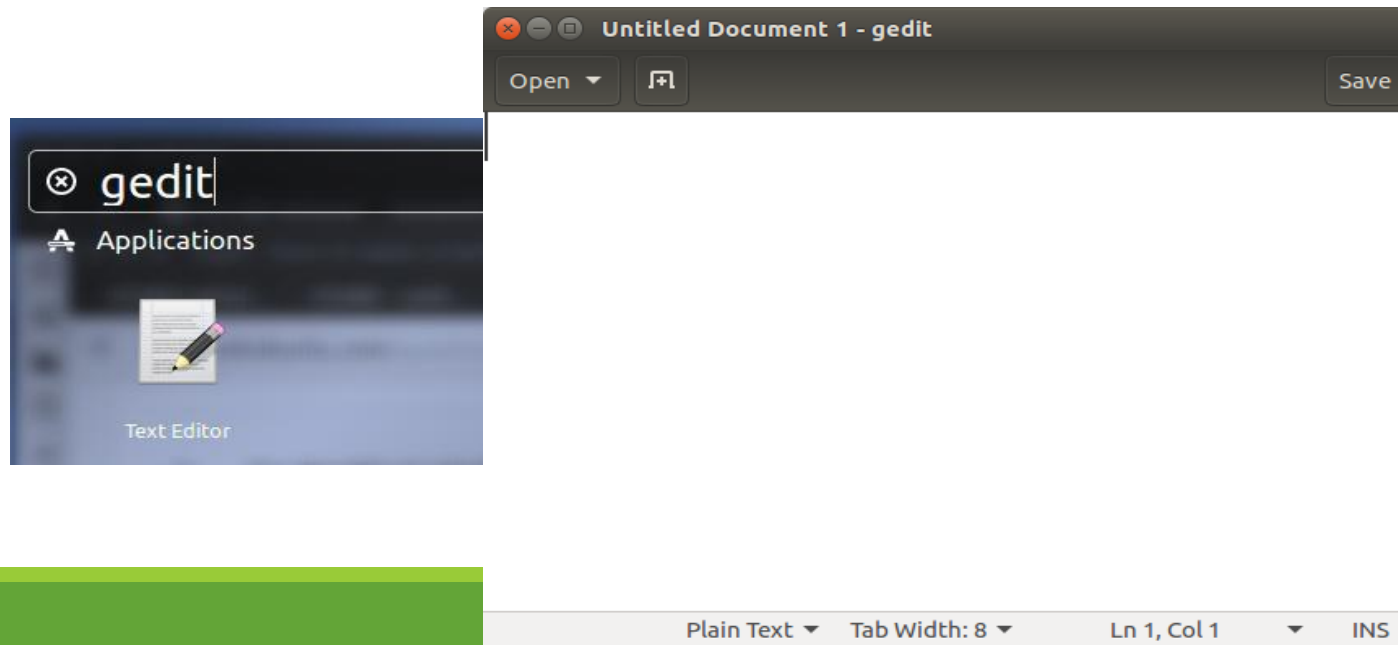
Terminal

```
bilg@bilg: ~  
bilg:~$ ls -a  
.  
..  
.adobe  
.bash_history  
.bash_logout  
.bashrc  
.cache  
.cdemu-daemon.log  
.compiz  
.config  
.dbus  
Desktop  
.gksu.lock  
.gnome2  
.gnome2_private  
.gnupg  
.hplip  
.httrack.ini  
.ICEauthority  
.java  
.local  
.macromedia  
.mozilla  
Music  
.PlayOnLinux  
PlayOnLinux's virtual drives  
.profile  
Program  
Public  
.sudo_as_admin_successful  
Sysprog  
system.html  
Templates  
.thunderbird  
Videos  
websites  
  
bilg@bilg: ~  
bilg:~$ ls -l  
total 144  
drwxr-xr-x  2 bilg bilg  4096 $ub  3 12:39 Desktop  
drwxr-xr-x  3 bilg bilg  4096 $ub  3 11:33 Documents  
drwxr-xr-x  2 bilg bilg  4096 $ub  3 11:02 Downloads  
-rw-r--r--  1 bilg bilg  8980 0ca 12 18:44 examples.desktop  
drwxr-xr-x  2 bilg bilg  4096 0ca 12 20:48 Music  
drwxr-xr-x  2 bilg bilg  4096 0ca 29 12:08 Pictures  
lrwxrwxrwx  1 bilg bilg    36 0ca 13 18:29 PlayOnLinux's virtual  
drives -> /home/bilg/.PlayOnLinux//wineprefix/  
drwxrwxr-x  2 bilg bilg  4096 0ca 13 18:49 Program  
drwxr-xr-x  2 bilg bilg  4096 0ca 12 20:48 Public
```

Editor Name	Features	Link
Code::Blocks	Open source, multi-compiler support, extensible with plugins.	http://www.codeblocks.org/
CLion (JetBrains)	Smart code assistance, powerful debugging, CMake integration.	https://www.jetbrains.com/clion/
Eclipse CDT	Eclipse's C/C++ plugin; large projects, extensive features.	https://www.eclipse.org/cdt/
NetBeans	Open source; C/C++ support, GUI design, project management.	https://netbeans.apache.org/
CodeLite	Lightweight, project management capabilities, debugging support.	https://codelite.org/
Visual Studio Code	Can be used like an IDE with extensions; excellent C/C++ support.	https://code.visualstudio.com/
Xcode	Apple's IDE; good for macOS/iOS development, GUI and console programs.	https://developer.apple.com/xcode/
Dev-C++	A lightweight IDE especially for Windows; suitable for beginners.	https://sourceforge.net/projects/orwelldevcpp/
Qt Creator	Very suitable for GUI development with Qt; cross-platform support.	https://www.qt.io/qt-creator
Geany	A lightweight editor that consumes fewer resources, opens quickly, and has project support.	https://www.geany.org/
Vim / Neovim	Modal structure, keyboard shortcuts, powerful plugin support.	https://vim.org/ ; https://neovim.io/
Emacs	Highly customizable, extensive plugin and package ecosystem.	https://www.gnu.org/software/emacs/
Atom	Easy user interface, extensible with plugins; has former popularity.	https://atom.io/
Notepad++	Lightweight, Windows-focused; good syntax highlighting and plugins for C/C++ files.	https://notepad-plus-plus.org/
Textadept	A minimal structure; extensible with Lua; both GUI and terminal modes.	https://orbitalquark.github.io/textadept/

gedit

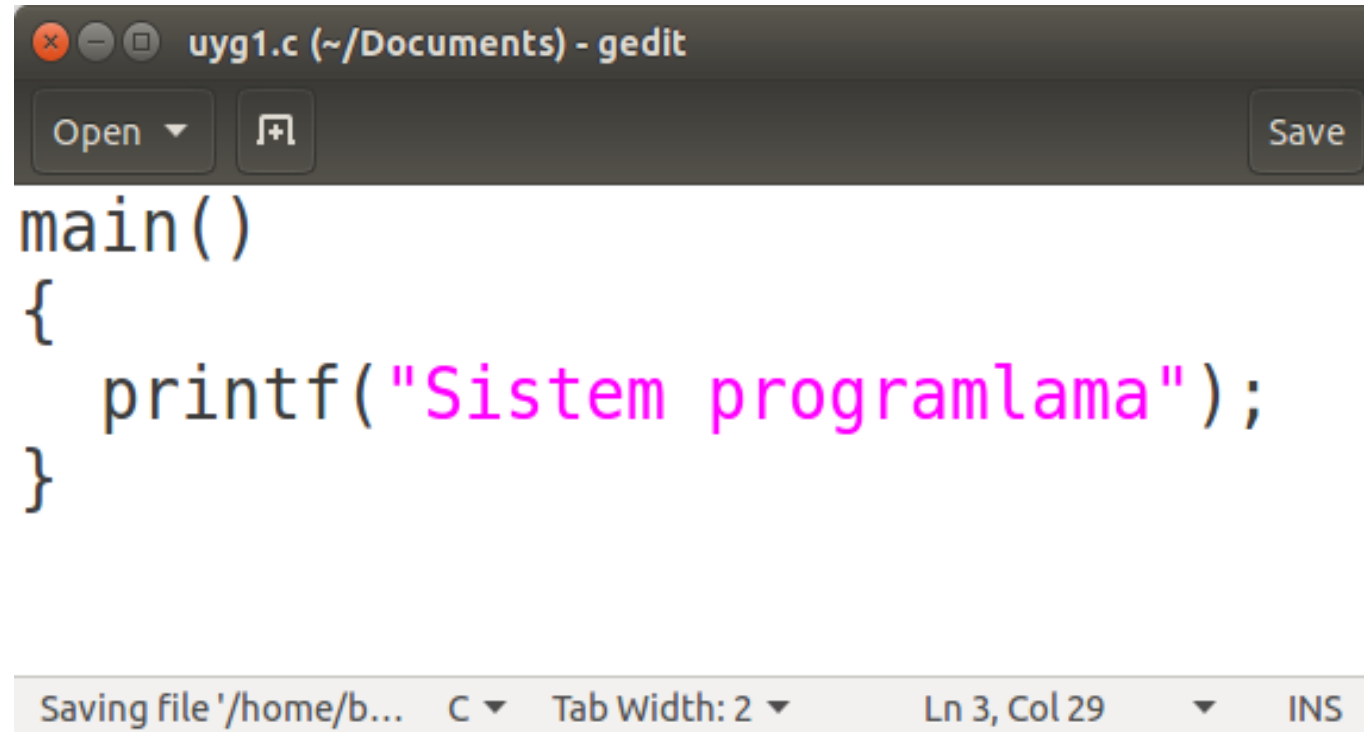
gedit is the default text editor for the GNOME desktop environment on Linux. It is a simple, lightweight, and user-friendly editor designed for general text editing. gedit supports syntax highlighting for many programming languages, including C, C++, Python, and HTML, making it useful for both coding and writing. It also offers features like search and replace, line numbering, auto-indentation, and plugins for extended functionality. Because of its simplicity, gedit is often preferred by beginners and users who need a clean and distraction-free editor.



Example C Program

Let's write the following program using the editor.

The sample program simply prints a message to the screen.



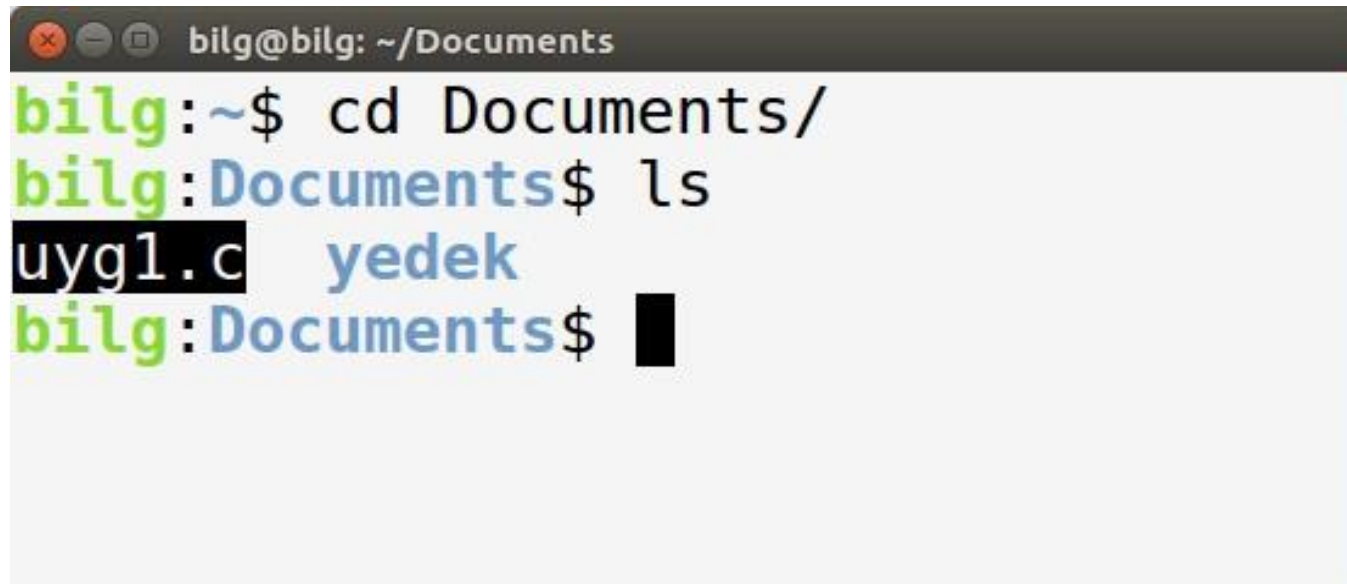
The image shows a screenshot of a gedit text editor window. The title bar at the top reads "uyg1.c (~/Documents) - gedit". Below the title bar, there is a toolbar with an "Open" button, a file icon, and a "Save" button. The main editing area contains the following C code:

```
main()  
{  
    printf("Sistem programlama");  
}
```

At the bottom of the window, a status bar displays the text "Saving file '/home/b..." followed by a dropdown menu showing "C", "Tab Width: 2", "Ln 3, Col 29", and "INS".

Compiling the Program

- After saving the Uyg1.c file, let's compile it using the GNU C Compiler.
-
- gcc comes built into Ubuntu.
 - First, let's change the current directory to the folder containing the Uyg1.c file.

A terminal window with a dark title bar showing 'bilg@bilg: ~/Documents'. The terminal text is as follows:

```
bilg:~$ cd Documents/  
bilg:Documents$ ls  
uyg1.c  yedek  
bilg:Documents$
```

The terminal output shows the user 'bilg' changing the directory to 'Documents/' and then listing the contents with 'ls'. The files 'uyg1.c' and 'yedek' are listed. The prompt returns to 'bilg:Documents\$'.

Program Compilation

- After writing `gcc` as shown below and specifying the file name, our file will be compiled.
- The compilation result is related to adding the names of the libraries used, based on the output.

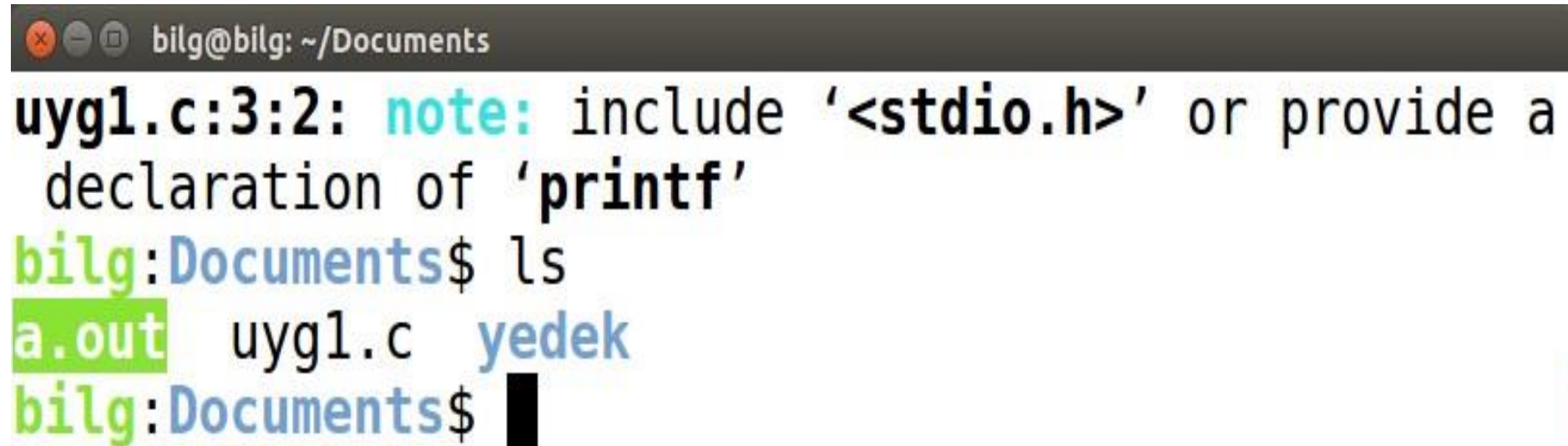
```
bilg@bilg: ~/Documents
bilg:Documents$ gcc uyg1.c
uyg1.c:1:1: warning: return type defaults to 'int'
[-Wimplicit-int]
main()
^

uyg1.c: In function 'main':
uyg1.c:3:2: warning: implicit declaration of function 'printf'
[-Wimplicit-function-declaration]
printf("Sistem programlama");
^

uyg1.c:3:2: warning: incompatible implicit declaration of built-in
function 'printf'
uyg1.c:3:2: note: include '<stdio.h>' or provide a declaration of
'printf'
bilg:Documents$
```

Program Compilation

If we type `ls` again from the command window and look at the files in the folder, we can see that a file named `a.out` has been created as a result of the compilation.

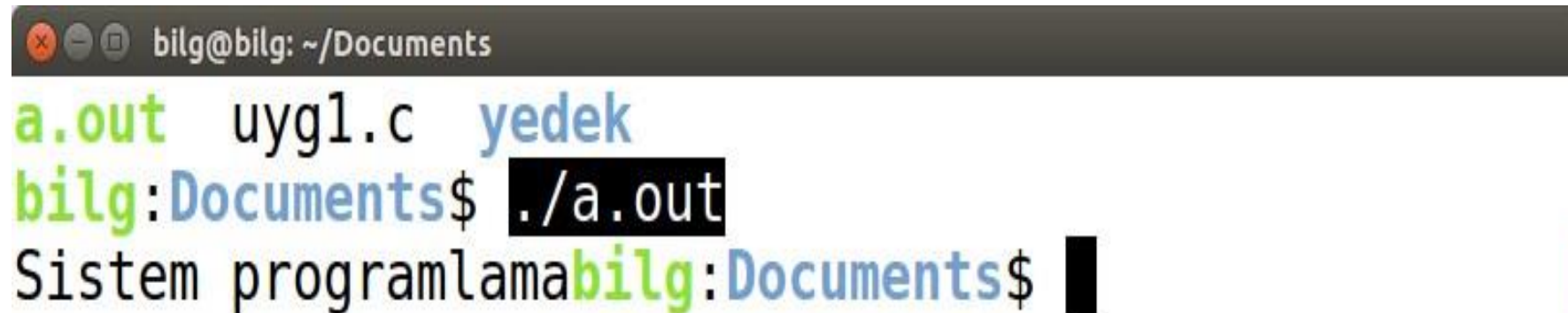


```
bilg@bilg: ~/Documents
uyg1.c:3:2: note: include '<stdio.h>' or provide a
  declaration of 'printf'
bilg:Documents$ ls
a.out  uyg1.c  yedek
bilg:Documents$
```

The image shows a terminal window with a dark title bar containing window control icons and the text 'bilg@bilg: ~/Documents'. The terminal output shows a compilation error for 'uyg1.c' at line 3, column 2, suggesting the inclusion of '<stdio.h>' or a declaration for 'printf'. Following this, the user runs the 'ls' command, and the output lists three files: 'a.out' (highlighted in green), 'uyg1.c', and 'yedek'. The prompt returns to 'bilg:Documents\$' with a black cursor.

Running the Program

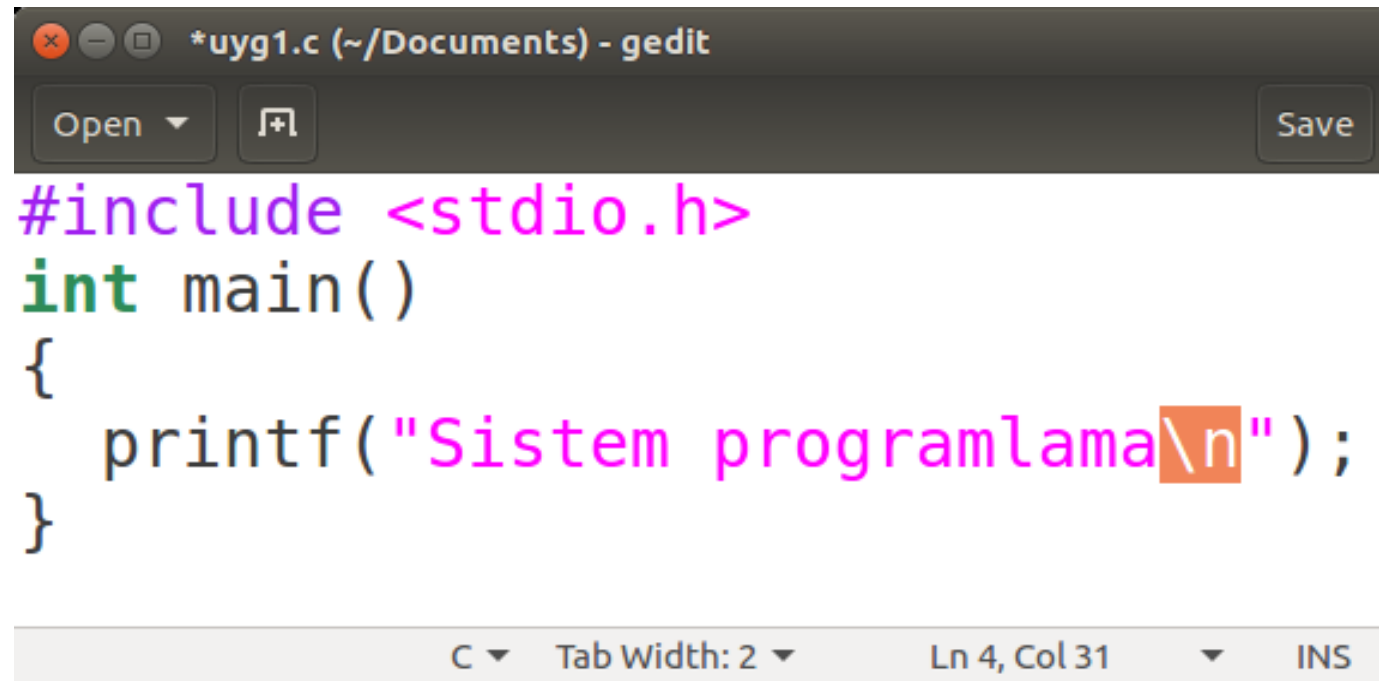
If we type ls again from the command window and look at the files in the folder, we can see that a file named a.out has been created as a result of the compilation.

A terminal window with a dark title bar containing window control icons and the text 'bilg@bilg: ~/Documents'. The terminal shows a file listing with 'a.out' in green, 'uyg1.c' in white, and 'yedek' in blue. Below this, the prompt 'bilg:Documents\$' is followed by the command './a.out' which is highlighted with a black background. The next line shows the output 'Sistem programlama' followed by the prompt 'bilg:Documents\$' and a black cursor block.

```
bilg@bilg: ~/Documents
a.out  uyg1.c  yedek
bilg:Documents$ ./a.out
Sistem programlama bilg:Documents$
```

Remove compiling warning

To remove compilation warnings, let's add the `stdio.h` library to the program, which provides the `printf` function, and change the return type of the `main` function to `int`.

A screenshot of a gedit text editor window. The title bar shows the file name *uyg1.c (~/.Documents) - gedit. The editor contains C code: #include <stdio.h>, int main(), {, printf("Sistem programlama\n");, and }. The status bar at the bottom indicates the file is in C mode, with a tab width of 2, and the cursor is at line 4, column 31 in insert mode.

```
*uyg1.c (~/.Documents) - gedit
Open Save
#include <stdio.h>
int main()
{
    printf("Sistem programlama\n");
}
C Tab Width: 2 Ln 4, Col 31 INS
```

Rename the compiling program

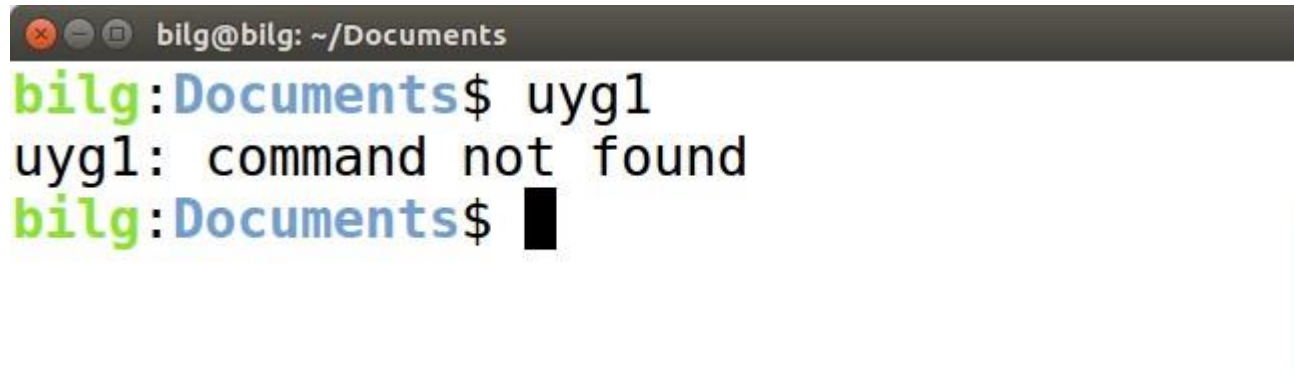
To change the name of the program we compiled, the -o option is used as follows

`gcc test.c -o programadi`

```
bilg@bilg: ~/Documents
bilg:Documents$ ./a.out
Sistem programlama
bilg:Documents$ gcc uyg1.c -o uyg1
bilg:Documents$ ls
a.out  uyg1  uyg1.c  yedek
bilg:Documents$ ./uyg1
Sistem programlama
bilg:Documents$ gcc -o uyg1 uyg1.c
bilg:Documents$ ./uyg1
Sistem programlama
bilg:Documents$
```

System path - PATH variable

The `./` we write before the compiled file when running the program prevents it from getting mixed up with other programs in the system path.

A terminal window with a dark title bar containing the text 'bilg@bilg: ~/Documents'. The terminal shows a prompt 'bilg:Documents\$' followed by the command 'uyg1'. The next line shows the error message 'uyg1: command not found'. The prompt 'bilg:Documents\$' is shown again on the next line, followed by a black cursor block. A vertical orange line is positioned to the right of the terminal window.

```
bilg@bilg: ~/Documents
bilg:Documents$ uyg1
uyg1: command not found
bilg:Documents$ █
```

If we want to run the program without using `./`, we need to add the directory/folder where the compiled code is located to the system path."

System Path variable

To see the directories in the system path, we can print the PATH variable with the echo command.

```
bilg@bilg: ~/Documents
bilg:Documents$ echo $PATH
/home/bilg/bin:/home/bilg/.local/bin:/usr/local/s
bin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:
/usr/games:/usr/local/games:/snap/bin
bilg:Documents$
```

The `pwd` command, which stands for "print working directory," is a fundamental command-line utility in Unix-like operating systems (such as Linux and macOS). Its primary function is to display the full, absolute path of the current working directory.

```
bilg@bilg: ~/Documents
bilg:Documents$ pwd
/home/bilg/Documents
bilg:Documents$
```

```
bilg@bilg: ~/Documents
bilg:Documents$ pwd
/home/bilg/Documents
bilg:Documents$ PATH=$PATH:/home/bilg/Documents
bilg:Documents$ echo $PATH
/home/bilg/bin:/home/bilg/.local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/home/bilg/Documents
bilg:Documents$
```

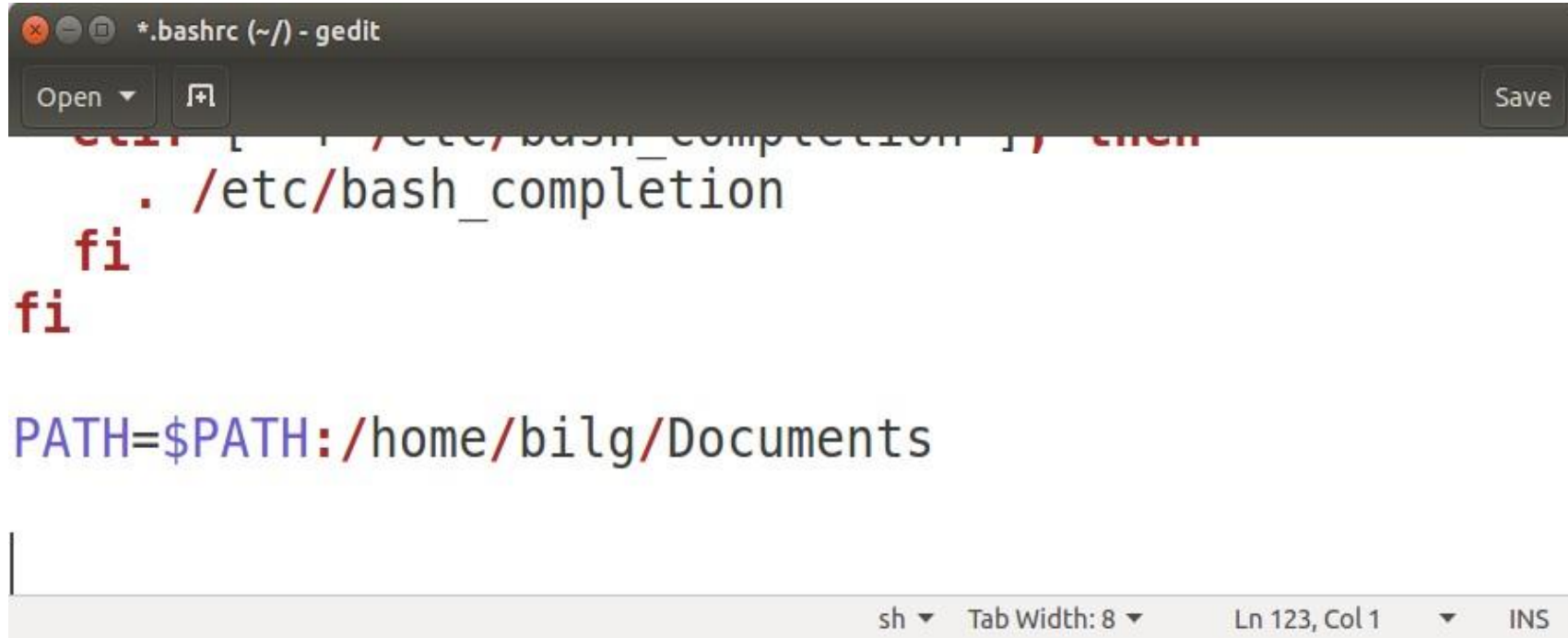
Using the pwd command directly instead of write the path add the currently directory to system path.

```
bilg@bilg: ~/Documents
bilg:Documents$ PATH=$PATH:/home/bilg/Documents
bilg:Documents$ PATH=/home/bilg/Documents:$PATH
bilg:Documents$ PATH=$PATH:$(pwd)
bilg:Documents$ PATH=$(pwd):$PATH
bilg:Documents$
```

When we add the program to the system path, we can run it by directly typing its name without using ./

```
bilg@bilg: ~/Downloads
bilg:Documents$ PATH=$PATH:$(pwd)
bilg:Documents$ uyg1
Sistem programlama
bilg:Documents$ cd ..
bilg:~$ uyg1
Sistem programlama
bilg:~$ cd Downloads/
bilg:Downloads$ uyg1
Sistem programlama
bilg:Downloads$
```

- The changes we make are lost when we close and reopen the terminal window.
 - To make them permanent, we can use the ~/.bashrc file.
 - From the command line: gedit ~/.bashrc
-



The screenshot shows a gedit editor window titled "*.bashrc (~/) - gedit". The window has a dark theme. The menu bar includes "Open" and "Save" buttons. The main text area contains the following code:

```
if [ -f /etc/bash_completion ] ; then
. /etc/bash_completion
fi
fi

PATH=$PATH:/home/bilg/Documents
```

The status bar at the bottom indicates "sh", "Tab Width: 8", "Ln 123, Col 1", and "INS".

whereis

The whereis command is a utility in Unix-like operating systems (such as Linux and macOS) used to locate the binary, source, and manual page files for a specified command.

It's particularly useful for understanding where a program's components are installed on your system.

* whereis searches for executable files, their source code, and their manual pages in a predefined list of standard locations. It's often used by system administrators and developers to quickly find the paths to commands.

```
bilg@bilg: ~/Documents
bilg:Documents$ ./test
Sistem programlama
bilg:Documents$ test
bilg:Documents$ whereis test
test: /usr/bin/test /home/bilg/Documents/test
/usr/share/man/man1/test.1.gz
bilg:Documents$
```

